

Temas Selectos de Física Computacional III:

Introducción a la ciencia de datos.

Clase 7: 18 Marzo 2025

Dr. Leonid Serkin

1 Introducción

En esta clase, estudiaremos los problemas de clasificación y cómo pueden abordarse utilizando una categoría de modelos basados en árboles de decisión de Scikit-learn, usando el conjunto de datos del vino.

2 Aprendizaje supervisado y no supervisado

El entrenamiento de algoritmos de aprendizaje automático (machine learning) se subdivide en lo que se denomina aprendizaje **supervisado** y **no supervisado**.

El aprendizaje supervisado trata con problemas donde las clases de los objetos en el conjunto de entrenamiento son conocidas *a priori*; por lo tanto, se dice que el conjunto de entrenamiento está **etiquetado**.

En el aprendizaje no supervisado, las clases de los objetos en el conjunto de entrenamiento *no se conocen a priori*, y se dice que el conjunto de entrenamiento **no está etiquetado**.

Recordando el ejemplo de reconocimiento de escritura a mano, las imágenes de dígitos escritos a mano deben ser colocadas en una de diez clases. Al aplicar el aprendizaje supervisado, requerimos un conjunto de entrenamiento etiquetado. Las etiquetas (es decir, las clases) son, por ejemplo, “0,” “1,”... “9” para los dígitos. Nuestro conjunto de entrenamiento es una colección de imágenes junto con la etiqueta correspondiente a cada imagen.

Al entrenar un algoritmo de aprendizaje automático en este entorno **supervisado**, se pueden comparar las salidas del algoritmo con las etiquetas que se sabe que son correctas.

Cuando la salida del algoritmo no coincide con la etiqueta correcta, este se mejora a sí mismo (“**aprende**”) y, con suerte, producirá resultados más precisos en el futuro. Una vez que se ha completado el entrenamiento, los parámetros del algoritmo se fijan y este algoritmo puede ser utilizado en una gama más amplia de imágenes.

En contraste, los algoritmos de aprendizaje **no supervisado** aprenden de conjuntos de entrenamiento no etiquetados. En el ejemplo anterior, esto significaría que nuestro conjunto de entrenamiento contiene solo una colección de imágenes sin saber a qué dígito corresponde cada imagen. En lugar de clasificación, a veces se refiere a esto como un problema de "clustering". En algunos casos, todavía sabemos cuántos grupos formar. Por ejemplo, si sabemos que tenemos imágenes de dígitos escritos a mano, también sabemos que debemos obtener diez grupos; pero en otras situaciones, incluso el número de clases es desconocido.

3 Conjuntos de datos del vino

El conjunto de datos del vino es un conjunto de datos famoso disponible en el repositorio <https://archive.ics.uci.edu/dataset/109/wine>. Los datos son el resultado de un **análisis químico de vinos** cultivados en la misma región en Italia, pero provenientes de **tres tipos diferentes de cultivos** (variedades de uva). El análisis se realizó para determinar la concentración de 13 componentes químicos que son característicos de los vinos. Los 13 atributos que componen el conjunto de datos son:

1. Alcohol: Cantidad de alcohol presente en el vino (porcentaje).
2. Malic acid: Contenido de ácido málico, un ácido orgánico que se encuentra naturalmente en las uvas.
3. Ash: Cantidad de ceniza (residuos sólidos) presente en el vino, lo que refleja los minerales presentes.
4. Alkalinity of ash: Alcalinidad de la ceniza, relacionada con la acidez del vino.
5. Magnesium: Cantidad de magnesio, un mineral esencial que afecta la calidad del vino.
6. Total phenols: Total de fenoles, compuestos que afectan el sabor, color y sensación del vino.
7. Flavanoids: Un tipo de fenol que tiene propiedades antioxidantes y es clave para el sabor y color.
8. Nonflavanoid phenols: Fenoles no flavonoides, otro tipo de compuesto fenólico que también influye en las características del vino.
9. Proanthocyanins: Compuestos antioxidantes que están relacionados con el color y la astringencia del vino.
10. Color intensity: Intensidad del color del vino, relacionada con la cantidad de pigmentos presentes.
11. Hue: Tono o color del vino, otro factor importante en la percepción visual.

12. OD280/OD315 of diluted wines: Relación óptica entre los valores de absorbancia a 280 nm y 315 nm, utilizada para medir las proteínas y otros compuestos en el vino.
13. Proline: Aminoácido presente en el vino que puede estar asociado con la calidad del vino y su sabor.

El objetivo principal de este dataset es **clasificar** los vinos en tres categorías (basado en las características químicas). A partir de estos 13 componentes (características), tu objetivo es predecir la clase objetivo (target). Puedes descargar el conjunto de datos desde UCI o puedes obtenerlo directamente desde Scikit-learn:

```
import pandas as pd
from sklearn.datasets import load_wine
dataset = load_wine()
df = pd.DataFrame(data=dataset["data"], columns=dataset["feature_names"])
df["target"] = dataset["target"]
print(df.head())
```

Carga el conjunto de datos desde scikit-learn. Luego:

- Responde a las siguientes preguntas: Cuántos registros están disponibles? Hay valores faltantes? De que tipo son las variables? Cual es sus valores medios y otros de análisis de estadística básica?
- Cuántos elementos contiene cada clase de "target"? Es un problema balanceado?
- Genera una matriz de gráficos de dispersión (scatter plots) utilizando la librería Seaborn.

```
import seaborn as sns
import matplotlib.pyplot as plt
custom_palette = {0: "red", 1: "blue", 2: "green"}
sns.pairplot(df, hue="target", palette=custom_palette)
plt.show()
```

- Estudia las correlaciones entre las variables usando `sns.heatmap(df.corr())`

4 Árboles de decisión

Los árboles de decisión son algoritmos de aprendizaje **supervisado** utilizados tanto para tareas de clasificación como de regresión.

La idea principal de los árboles de decisión es encontrar aquellas características descriptivas que contienen la mayor cantidad de **información** con respecto a la

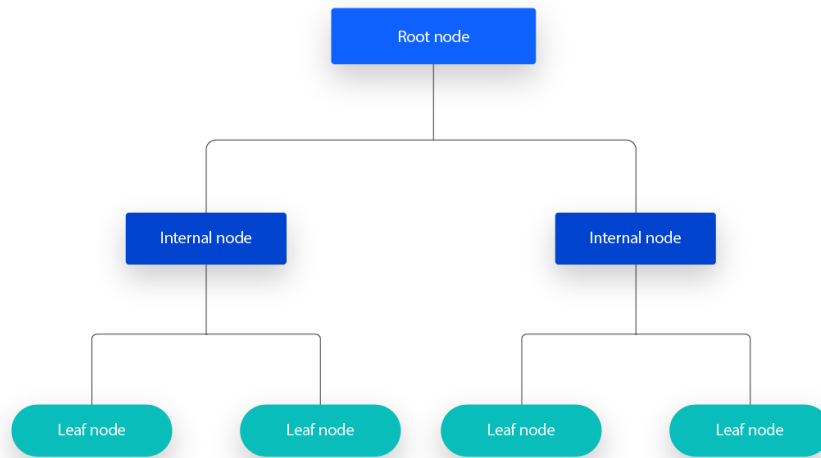


Figure 1: La idea básica de un árbol de decisión.

característica objetivo, y luego dividir el conjunto de datos según los valores de estas características de tal manera que los valores de la característica objetivo para los subconjuntos resultantes sean lo más puros posible.

Las características descriptivas que mejor reproducen las características objetivo normalmente se consideran las más informativas. El proceso de encontrar la característica más informativa se lleva a cabo hasta que se cumple un criterio de detención, momento en el cual llegamos a los llamados **nodos hoja**.

Un árbol de decisión generalmente se divide en un **nodo raíz**, los nodos interiores y los nodos hoja finales, o simplemente **hojas**. Estas entidades están conectadas mediante las llamadas **ramas**.

Los nodos hoja contienen las predicciones que realizaremos para nuevas instancias de consulta presentadas a nuestro modelo entrenado. Esto es posible porque el modelo ha aprendido la estructura subyacente de los datos de entrenamiento y, por lo tanto, puede, dado ciertos supuestos, **hacer predicciones** sobre el valor de la característica objetivo (clase) de instancias de consulta no vistas.

El enfoque general de los árboles de decisión es un enfoque de arriba hacia abajo. Una instancia se clasifica comenzando en el nodo raíz del árbol, probando el atributo especificado por este nodo, y luego bajando por la rama del árbol que corresponde al valor del atributo en el ejemplo dado. Este proceso se repite para el subárbol enraizado en el nuevo nodo.

5 Construyendo el árbol de decisión

En este ejercicio utilizarás `DecisionTreeClassifier` de Scikit-learn para construir un árbol de decisión para el conjunto de datos del vino (<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>).

Primero, crearemos un objeto `DecisionTreeClassifier` con la configuración predeterminada (es decir, sin pasar ningún parámetro al constructor), y lo entrenamos usando las variables del conjunto de datos del vino que hemos definido.

```
from sklearn.tree import DecisionTreeClassifier
X = dataset["data"]
y = dataset["target"]
clf = DecisionTreeClassifier()
clf.fit(X, y)
```

Felicidades, has creado un árbol! Ahora vamos a visualizarlo. Scikit-learn ofrece una función `plot_tree()`, que muestra el árbol.

```
from sklearn.tree import plot_tree
plt.figure(figsize=(50, 20))
plot_tree(clf, feature_names=dataset["feature_names"],
          class_names=dataset["target_names"], filled=True)
plt.show()
```

Una alternativa es usar `export_graphviz()`. Esta función exporta el árbol como un archivo DOT, un lenguaje para describir gráficos. Desde el código DOT puedes generar la representación visual resultante con el siguiente fragmento de código:

```
from sklearn.tree import export_graphviz
import pydot
from IPython.display import Image
dot_code = export_graphviz(clf, feature_names=dataset["feature_names"],
                           class_names=dataset["target_names"])
graph = pydot.graph_from_dot_data(dot_code)
Image(graph[0].create_png())
```

Ahora si analizamos el árbol: fíjate en cómo **cada división genera dos ramas**: como se puede ver desde la división raíz, la convención utilizada es que la rama más a la izquierda es la que se selecciona cuando la comparación da un valor "**verdadero**", y la rama más a la derecha cuando la comparación es "**falsa**".

Cada uno de los nodos del árbol contiene cierta información, y puedes analizar el árbol en más detalle.

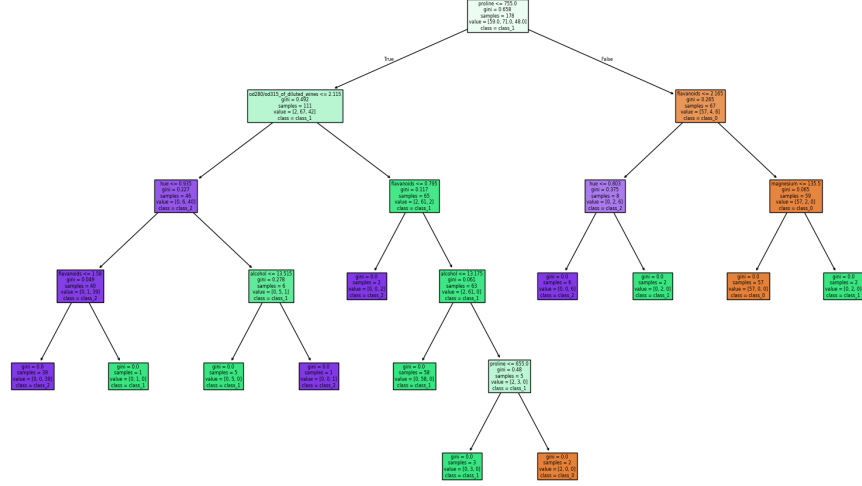


Figure 2: Tu árbol de decisión.

- El número de muestras que se encuentran en cada nodo. En el nodo raíz, comenzamos con 178 muestras, y en cada nodo se especifica cuántos puntos del conjunto de datos alcanzan ese nodo. `class = class_1`: este valor indica que el modelo ha asignado esta clase, en este caso, `class_1`, como la clase mayoritaria para el nodo raíz.
- El atributo utilizado para la división y el valor umbral usado en este primer nodo es `proline <= 755.0`.
- Se utiliza el **índice Gini** como un indicador de qué tan puro es cada nodo:

$$GINI = 1 - \sum_{i=1}^k p_i^2$$

donde k es el número de clases, p_i es la proporción de elementos que pertenecen a la clase i . En problemas binarios, el índice Gini varía entre 0 y 0.5. Para el caso de tres clases:

$$GINI = 1 - p_1^2 - p_2^2 - p_3^2$$

y el índice Gini máximo es 0.667. Para un problema de N clases, el índice Gini varía de 0 a $(1 - \frac{1}{N})$.

- Calculemos el índice Gini en el nodo raíz:

$$GINI = 1 - \left(\frac{59}{178}\right)^2 - \left(\frac{71}{178}\right)^2 - \left(\frac{48}{178}\right)^2$$

- Nodo raíz: `proline <= 755.0`. La condición es: si el valor de `proline` es menor o igual a 755, se sigue la rama izquierda; si es mayor a 755, se sigue la rama derecha. Vamos a observar la rama **derecha**, donde `proline > 755.0`.
- Siguiente nodo: `flavanoids <= 2.165`. La condición es: si el valor es menor o igual a 2.165, se sigue la rama izquierda de este nodo. Si es mayor que 2.165, se sigue la rama **derecha**.
- Siguiente nodo: `magnesium <= 135.5`. La condición es: si el valor es menor o igual a 135.5, se sigue la rama izquierda. Si es mayor que 135.5, se sigue la rama **derecha**.
- Nodo de hoja: `class = class_1`. Finalmente, llegamos a un nodo hoja con 2 muestras, donde todas las muestras pertenecen a la clase 1.
- En los nodos intermedios, el Gini es mayor que 0 porque hay mezcla de clases. En los nodos finales, el Gini es 0 porque solo hay una clase, lo que indica pureza total. Un árbol bien entrenado busca crear nodos lo más puros posible en cada paso para lograr una mejor clasificación.
- Nota que algunos de los nodos están "tomando decisiones" basadas en pequeñas porciones del conjunto de datos de entrenamiento. Por ejemplo, la división `flavonoides <= 1.58` separa 39 elementos de la clase "2" de un solo elemento de la clase "1".
- Observa que varias características se usan en múltiples ocasiones. Por ejemplo, la característica `flavonoides` se utiliza 3 veces en diferentes caminos de raíz a hoja, lo cual está bien, ya que el árbol no "descarta" las características que ya ha utilizado.
- La característica `alcohol` se usa dos veces en el mismo camino de raíz a hoja (`alcohol <= 13.175` y `alcohol <= 13.515`). En estos casos, los diferentes umbrales están definidos de manera compatible: dado que la división `alcohol <= 13.515` está a la derecha del nodo anterior, significa que el valor de `alcohol` ya es mayor que 13.175. Esto implica que los elementos en la hoja izquierda tendrán un valor de `alcohol` que está entre 13.175 y 13.515.

6 Predicciones

Ahora que hemos analizado visualmente el árbol de decisión, usa la función `predict()` de `DecisionTreeClassifier` para obtener las predicciones del clasificador. Luego, usa `accuracy_score()` para calcular la precisión entre la lista de etiquetas correctas y la lista de predicciones hechas por el clasificador.

El resultado es 1: es decir, 100% de precisión. La razón por la que estamos obteniendo una precisión tan alta es que estamos **probando nuestro modelo**

con los mismos datos que usamos para el entrenamiento. No es particularmente útil calcular la precisión con los mismos datos que usamos para el entrenamiento, ya que eso no nos dirá **nada sobre qué tan bien nuestro clasificador generaliza a nuevos datos**.

6.1 División del conjunto en train y test

Lo que vamos a hacer ahora es dividir el conjunto de datos en un conjunto de entrenamiento y uno de prueba usando `train_test_split()`, y evaluar cómo la forma en que se dividen los datos de un conjunto de clasificación afecta la distribución de clases y, en consecuencia, el rendimiento del modelo.

1. Divide los datos usando:

```
from sklearn.model_selection import train_test_split
```

Utiliza un 80% de los datos para entrenamiento y un 20% para prueba.

2. Las opciones del método admiten:

```
sklearn.model_selection.train_test_split(*arrays, test_size=None,
train_size=None, random_state=None, shuffle=True, stratify=None)
```

y vamos a probar tres maneras diferentes:

- 1) sin mezclar: `shuffle=False`
 - 2) con mezclar: `shuffle=True`
 - 3) con estratificación: `stratify=y`
3. Crea los sub-conjuntos `X_train`, `y_train`, `X_test` y `y_test`, respectivamente. Cuenta la cantidad de muestras de cada clase en los conjuntos de entrenamiento y prueba, y calcula los porcentajes de cada clase en ambos conjuntos, para las tres opciones de división.
 4. Entrena un modelo para cada una de las tres opciones y calcula la precisión. Compara y analiza estos valores con la precisión obtenida previamente. Cual es el valor más razonable? Por qué? ¿En qué casos es crítico utilizar `stratify=y` y cuándo no es tan importante?

6.2 Evaluación de métricas

Para calcular otras métricas (por ejemplo, precisión, recall y F1-score), se usan las funciones respectivas (`precision_score`, `recall_score`, `f1_score`). Ten en cuenta que, dado que estas tres métricas están basadas en una sola clase, puedes calcular el valor para una clase específica, agregar los resultados en un solo valor o recibir los resultados para todas las clases. Consulta el parámetro `average` en la documentación para obtener más información sobre esto.

También puedes usar la función `classification_report`, que devuelve varias métricas (incluyendo las mencionadas anteriormente) para cada una de las clases del problema.

6.3 Ajuste de parámetros

Hasta ahora, solo has utilizado árboles de decisión en su configuración "predeterminada" (es decir, árboles de decisión que usan la configuración por defecto). Sin embargo, el árbol de decisión predeterminado podría no ser la mejor opción en términos de rendimiento para ajustar nuestro conjunto de datos.

En este ejercicio, realizarás una "búsqueda en rejilla" (*grid search*): definirás un conjunto de configuraciones posibles y, para cada configuración, construirás un clasificador. Luego, probarás el rendimiento de cada clasificador e identificarás la configuración que produce el mejor modelo.

En la documentación oficial de `DecisionTreeClassifier`, puedes encontrar una lista de todos los parámetros que puedes modificar. Identifica algunos de los parámetros que, basándote en tu conocimiento teórico sobre árboles de decisión, podrían afectar el rendimiento del árbol. Para cada uno de estos parámetros, define un conjunto de valores posibles (la documentación oficial proporciona información adicional sobre los valores posibles que pueden usarse).

Por ejemplo, el siguiente parámetro `max_depth` define la profundidad máxima del árbol de decisión, y puede establecerse en `None` (es decir, sin límite de profundidad) o en valores como 2, 4, 8.

Lo que puedes hacer es construir un diccionario de parámetros (es decir, un diccionario donde las claves son los nombres de los parámetros y los valores son listas de valores candidatos). Utilizando la clase `ParameterGrid` que ofrece Scikit-learn, puedes generar una lista de todas las configuraciones posibles que se pueden obtener a partir del diccionario de parámetros.

El siguiente es un ejemplo:

```
from sklearn.model_selection import ParameterGrid
parametros = {
    "max_depth": [None, 2, 4, 8]
}
for configuracion in ParameterGrid(parametros):
    print(configuracion)
```

Para mejorar el rendimiento del árbol de decisión, puedes probar diferentes combinaciones de parámetros clave. Aquí te dejo una lista:

```
"max_depth": [2, 4, 6, 8, 10],
"min_impurity_decrease": [0, 0.01, 0.03, 0.07, 0.09, 0.11]
```

Identifica y estudia los parámetros y posibles valores para estos. Entrena un modelo separado y validar con datos de prueba. Para cada configuración, calcula la precisión resultante en los datos de prueba. Luego, encuentra la configuración de parámetros que tenga la mayor precisión.

6.4 Validación cruzada

Con el ejercicio anterior, hemos definido cuál debería ser la mejor configuración de hiperparámetros. También tenemos una estimación de qué tan buena es la mejor configuración en términos de rendimiento (la que obtuvimos en el conjunto de prueba). Sin embargo, este estimado es, en cierto sentido, **sesgado**: hemos seleccionado la configuración que mejor funciona en nuestro conjunto de prueba.

Un método popular es la **validación cruzada k-fold**. En este método, el conjunto de entrenamiento se divide en k particiones. Se utilizan $k - 1$ particiones para el entrenamiento y la partición restante para la validación. Este proceso se repite hasta que todas las particiones hayan sido utilizadas una vez para validación.

Scikit-learn ofrece la clase `KFold` para realizar la validación cruzada **k-fold**. Puedes utilizar esta clase de la siguiente manera:

```
from sklearn.model_selection import KFold
X_train_valid, X_test, y_train_valid, y_test = train_test_split(...)
kf = KFold(5)
for indices_t, indices_v in kf.split(X_train_valid):
    X_train = X_train_valid[indices_t]
    X_valid = X_train_valid[indices_v]
    y_train = y_train_valid[indices_t]
    y_valid = y_train_valid[indices_v]
```

Aquí `kf.split()` genera un conjunto de pares de índices, donde el primer valor de cada par contiene los índices que deben utilizarse para el entrenamiento, y el segundo valor contiene los índices usados para la validación.

Entonces, lo que tienes que agregar al código es:

- a) Entrena el modelo con la mejor configuración del ejercicio pasado.
- b) Evalúa en el conjunto de validación.
- c) Guarda las precisiones y el tamaño de cada fold.
- d) Calcula la precisión promedio ponderada.

Recuerda que la precisión final no se calcula simplemente como el promedio de las precisiones de cada fold, sino como un promedio ponderado (`np.average` con el parámetro `weights`), donde el peso es el número de elementos en cada partición.

Analiza tu resultado: si al realizar la búsqueda en rejilla (ParameterGrid) obtuviste una precisión máxima en el conjunto de prueba, pero con validación cruzada (k-fold) la precisión promedio fue diferente, ¿cuál es la razón de esta diferencia y cuál de las dos medidas es más confiable?

7 Un ejemplo visual

Este es un ejemplo muy visual de un conjunto de datos de viviendas, donde se crea un modelo de árboles de decisión que pueda distinguir las viviendas ubicadas en Nueva York de aquellas localizadas en San Francisco. Para ello vamos a estudiar esta pagina web:

<http://www.r2d3.us/una-introduccion-visual-al-machine-learning-1/>.